

中国科学院大学

《计算机组成原理(研讨课)》实验报告

姓名 周禹驰 学号 2024K8009929027 专业 计算机科学与技术
实验项目编号 2 实验名称 单周期 RISC-V 处理器设计

- 注 1: 撰写此 Word 格式实验报告后以 PDF 格式保存 SERVE CloudIDE 的 `/home/serve-ide/cod-lab/reports` 目录下(注意: reports 全部小写)。文件命名规则: `prjN.pdf`, 其中 `prj` 和后缀名 `pdf` 为小写, `N` 为 1 至 4 的阿拉伯数字。例如: `prj1.pdf`。PDF 文件大小应控制在 5MB 以内。此外, 实验项目 5 包含多个选做内容, 每个选做实验应提交各自的实验报告文件, 文件命名规则: `prj5-projectname.pdf`, 其中“-”为英文标点符号的短横线。文件命名举例: `prj5-dma.pdf`。具体要求详见实验项目 5 讲义。
- 注 2: 使用 `git add` 及 `git commit` 命令将实验报告 PDF 文件添加到本地仓库 master 分支, 并通过 `git push` 推送到实验课 SERVE GitLab 远程仓库 master 分支(具体命令详见实验报告)。
- 注 3: 实验报告模板下列条目仅供参考, 可包含但不限定如下内容。实验报告中无需重复描述讲义中的实验流程。

完整实验流程包括设计、开发、调试三个步骤:

- **设计步骤**为在代码编写前的准备阶段, 主要包括模块划分和模块结构设计, 并绘制设计图纸。
- **开发步骤**为在 S-IDE 集成开发环境中进行 Verilog/C 代码编写, 并实现完整功能。
- **调试步骤**包括本地仿真验证后推送至云平台进行 FPGA 原型验证, 以及后续迭代调试的全过程。

一、设计成果(对设计流程的总结。需附上设计结构图, 可用 PPT 或者在线绘图软件绘制逻辑结构框图后保存为 PDF, 再插入到 L^AT_EX 中)

1 单周期 RISC-V 处理器设计

本实验实现了一个单周期 RISC-V32 处理器, 支持 RV32I 基本整数指令集中的前 37 条非特权指令。处理器采用经典的五段式数据通路结构, 所有指令在一个时钟周期内完成取指、译码、执行、访存和写回操作。

1.1 整体架构设计

处理器的整体结构可划分为以下功能模块:

1. **取指模块 (Fetch)**: 负责从指令存储器读取指令并更新程序计数器 (PC)。PC 的下一状态由多路选择器根据当前指令类型选择: 顺序执行时 `PC+4`, 跳转时 `PC+offset (JAL/Branch)`, 间接跳转时使用寄存器值 (`JALR`)。
2. **译码模块 (Decode)**: 从 32 位指令字中提取 `opcode`, `rd`, `rs1`, `rs2`, `funct3`, `funct7` 等字段, 并根据指令类型生成相应的立即数 (`U-type`, `J-type`, `I-type`, `S-type`, `B-type` 五种格式)。同时生成 ALU 操作码 (`ALUop`)、移位操作码 (`Shifto`)、分支条件等控制信号。
3. **执行模块 (Execute)**: 包括 ALU 和移位器两个运算单元。ALU 支持 8 种运算 (`AND`, `OR`, `XOR`, `NOR`, 加法、减法、有符号比较 `SLT`、无符号比较 `SLTU`); 移位器支持 3 种移位 (逻辑左移 `SLL`、逻辑右移 `SRL`、算术右移 `SRA`)。两个单元输出通过选择器合并。
4. **访存模块 (Memory)**: 处理 `Load/Store` 指令。Load 模块根据 `funct3` 和地址低 2 位从 32 位读数据中提取字节/半字并进行符号/零扩展; Store 模块根据 `funct3` 和地址低 2 位生成 `Write_strb` 字节使能信号, 并将写入数据移位到正确位置。

5. **写回模块 (Writeback)**: 通过多路选择器从 ALU 结果、移位结果、Load 数据、立即数 (LUI/AUIPC)、PC+4(JAL/JALR)中选择正确的写回数据写入寄存器文件。

电路结构图如下:

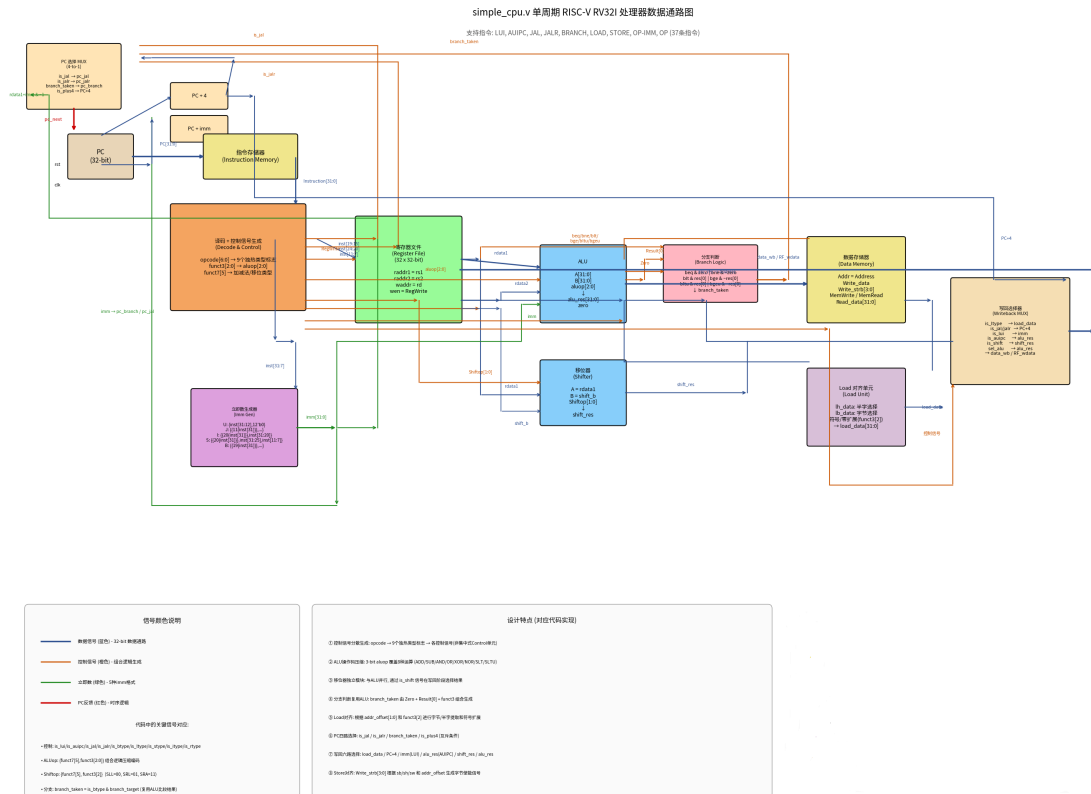


图 1: 电路结构图

1.2 指令编码压缩方案

本设计重点在于将 RV32I 的 37 条指令所需的控制信号通过组合逻辑进行压缩编码, 而非使用逐条译码的查找方式, 具体压缩策略如下:

第一级压缩: opcode 分组 6 位 opcode 的编码空间为 64, 但 RV32I 非特权指令仅使用 9 种 opcode。本设计将 9 种 opcode 映射为 9 个类型标志信号 (is_lui, is_auipc, is_jal, is_jalr, is_btype, is_ltype, is_stype, is_itype, is_rtype), 使用 9 路相等比较器实现。这 9 个信号作为后续所有控制逻辑的基础, 将 6 位 opcode 信息压缩为独热码形式。

第二级压缩: ALU 操作码编码 37 条指令中, 大部分需要通过 ALU 完成运算。本设计将所有 ALU 操作压缩为 3 位 ALUOp 信号, 支持 8 种运算。编码规则如下:

- **加法类操作**: R-type ADD(func7[5]=0) 编码为 3'b010; SUB(func7[5]=1) 编码为 3'b110。I-type ADDI 固定为 3'b010。编码逻辑: {func7[5], 2'b10}, 利用 func7 的最高位区分加/减法。
- **逻辑运算**: 将 func3 的三位编码映射为 ALUOp。以 XOR (func3=3'b100)、OR (func3=3'b110)、AND (func3=3'b111) 为例, 使用异或逻辑 {func3[2]^func3[1], 1'b0, func3[1]^func3[0]} 实现映射: XOR→100、OR→001、AND→000。这种编码方法与 ALU 内部的功能选择信号直接对应。

- **比较运算**: SLT (funct3=3'b010) 和 SLTU (funct3=3'b011) 编码为 {~funct3[0], 2'b11}, 即 SLT→111、SLTU→011。ALU 内部通过 ALUop[2:1] 选择比较模式(有符号/无符号), ALUop[0] 恒为 1 使 ALU 执行减法操作。
- **分支指令**: 6 条分支指令 (BEQ、BNE、BLT、BGE、BLTU、BGEU) 均通过 ALU 做减法后判断结果。ALUop 编码为 {~funct3[1], 1'b1, funct3[2]}。分支方向判断使用独立的组合逻辑, 将 ALU 的 Zero 和 Result[0] 与 funct3 组合产生 branch_taken 信号。
- **默认处理**: 非上述类型的指令 (LUI、AUIPC、JAL、JALR、Load、Store) 默认 ALUop 为 3'b010, 执行加法运算(计算地址或立即数)。

第三级压缩: 移位操作编码 移位指令 (SLL、SRL、SRA 及其立即数版本 SLLI、SRLI、SRAI) 通过 is_shift 信号从 ALU 通路中分离, 单独路由至移位器模块。Shiftop 编码为 {funct7[5], funct3[2]}: SLL 为 2'b00, SRL 为 2'b01, SRA 为 2'b11。这种编码将 6 条移位指令压缩为 2 位控制信号。

压缩效果总结 通过三级压缩编码, 本设计使用以下控制信号覆盖了 37 条指令的全部执行需求:

- 9 位类型标志(独热码, 由 opcode 生成)
- 3 位 ALUop(由 funct3/funct7 组合逻辑生成)
- 2 位 Shiftop(由 funct3/funct7 字段直接映射)
- 6 位分支类型标志(由 opcode+funct3 生成)
- 3 位 Load/Store 宽度标志(由 opcode+funct3 生成)

2 关键模块设计

2.1 ALU 模块设计

ALU 模块是执行单元的核心, 支持 8 种运算操作。模块接口如下:

- 输入: 32 位操作数 A 和 B, 3 位 ALUop 控制信号
- 输出: 32 位 Result, 溢出标志 Overflow, 进位标志 CarryOut, 零标志 Zero

ALU 内部采用加法器作为核心计算单元, 通过选择输入操作数实现加法和减法:

1. **操作数预处理**: 根据 ALUop[2] 和 ALUop[0] 决定 B 是否取反(~B), 同时设置进位输入 Cin。对于减法操作(ALUop[2]=1 或 ALUop[0]=1), 执行 $A + \sim B + 1 = A - B$ 。
2. **溢出检测**: 当 A 和 B 同号但结果与 A 异号时判定为溢出, 即 $Overflow = (A[31] == Signed_B[31]) \&\& (A[31] \wedge add_res[31])$ 。
3. **比较运算**: SLT(有符号比较)取加法器结果的最高位异或溢出标志作为符号比较结果; SLTU(无符号比较)取进位输出作为无符号比较结果。
4. **结果选择**: 使用独热选择信号将 8 种运算结果合并为最终输出。每种选择信号由 ALUop 译码产生(如 sel_and = ALUop==3'b000), 通过位与-位或方式实现多路选择。
5. **零标志生成**: 对 Result 进行缩位或非运算, 当 Result 全零时 Zero=1。

2.2 移位器模块设计

移位器支持三种移位操作:逻辑左移(SLL)、逻辑右移(SRL)、算术右移(SRA)。

1. **算术右移实现**:首先将 32 位操作数 A 符号扩展为 64 位($\{32\{A[31]\}\}$, A),然后对 64 位数进行逻辑右移 B 位,最后取低 32 位。扩展的符号位在右移时自然填充到高位,实现了算术右移的效果。
2. **移位操作选择**:2 位 Shiftop 编码(SLL=00, SRL=01, SRA=11)通过独热选择合并左移、逻辑右移和算术右移的结果。

2.3 寄存器文件模块设计

寄存器文件包含 32 个 32 位通用寄存器,x0 寄存器硬连线为 0。

1. **写操作**:在时钟上升沿,当写使能 wen 有效且目标寄存器不是 x0(waddr != 0)时,将 wdata 写入 rf[waddr]。
2. **读操作**:组合逻辑读取,当读地址为 0 时返回 0(x0 恒为零),否则返回对应寄存器值。

3 多周期 RISC-V 处理器设计(如有)

本实验仅完成单周期处理器设计,未涉及多周期处理器部分。

二、 开发成果(对代码开发流程的总结。需附上包含注释的 Verilog HDL 关键代码段、相应信号的仿真波形和信号变化的说明)

1 单周期 RISC-V 处理器设计

1.1 取指与 PC 更新逻辑

取指阶段的核心是 PC 的下一状态选择。以下为关键代码:

```
assign pc_plus4 = pc + 32'd4;
assign pc_branch = pc + imm;
assign pc_jal = pc + imm;
assign pc_jalr = (rdata1 + imm) & ~32'd1; // 最低位清零保证对齐
assign is_puls4 = (~is_jal) & (~is_jalr) & (~branch_taken);
assign pc_next = ({32{is_jal}} & pc_jal) |
                 ({32{is_jalr}} & pc_jalr) |
                 ({32{branch_taken}} & pc_branch) |
                 ({32{is_puls4}} & pc_plus4);
```

PC 的四种更新路径通过独热选择信号合并:JAL 跳转、JALR 间接跳转、分支跳转和顺序执行。JALR 指令要求目标地址的最低位清零以保证指令对齐(RISC-V 规范要求)。

1.2 立即数生成

RISC-V 指令集定义了五种立即数格式,本设计通过 opcode 判断指令类型后生成对应的立即数:

```
assign imm = ({32{uimm}} & {Instruction[31:12],12'b0}) | // U-type
             ({32{jaliimm}} & {{11{Instruction[31]}},Instruction[31],
             Instruction[19:12],Instruction[20],
             Instruction[30:21],1'b0}) | // J-type
```

```

({32{jalrimm}} & {{20{Instruction[31]}}, Instruction[31:20]}) | // I-type
({32{stimm}} & {{20{Instruction[31]}}, Instruction[31:25],
Instruction[11:7]}) | // S-type
({32{bimm}} & {{19{Instruction[31]}}, Instruction[31],
Instruction[7], Instruction[30:25],
Instruction[11:8], 1'b0}) | // B-type
32'd0;

```

五种立即数的位排列在 RISC-V 指令编码中有意设计成交叉但规整的结构: I-type 的 `imm[11:0]` 位于指令的 31:20 位; S-type 的 `imm[11:5]` 位于 31:25 位、`imm[4:0]` 位于 11:7 位; B-type 的 `imm[12||10 : 5||4 : 1||11]` $U - type$ `imm[31 : 12]` $31 : 12$ $J - type$ `imm[20||10 : 1||11||19 : 12]` $B - type$

1.3 ALU 操作数与操作码生成

ALU 的输入操作数 A 和 B 根据指令类型选择不同的来源:

```

assign a_pc = is_auipc | is_jal; // AUIPC和JAL使用PC值
assign b_reg = (is_rtype & ~is_shift) | is_btype; // R-type和branch用寄存器
assign b_imm = (is_itype & ~is_shift) | is_ltype | is_stype
              | is_auipc | is_jalr; // I-type和访存用立即数
assign alu_a = ({32{a_pc}} & pc) | ({32{~a_pc}} & rdata1);
assign alu_b = ({32{b_reg}} & rdata2) | (({32{b_imm}} & imm))
              | ({32{sel_b_zero}} & 32'd0);

```

ALU 操作码 `ALUop` 的生成是压缩编码的核心, 其逻辑已在设计部分详述。此处补充条件优先级处理: `is_def` 信号覆盖所有不需要特殊 ALU 操作的指令类型 (LUI/JAL/JALR/Load/Store 的地址计算), 使这些指令默认执行加法。

1.4 分支判断逻辑

分支指令通过 ALU 计算两操作数的差值, 然后根据 `funct3` 和 ALU 结果标志判断是否跳转:

```

assign branch_target = (beq & zero) | (bne & ~zero) |
                      (blt & alu_res[0]) | (bge & ~alu_res[0]) |
                      (bltu & alu_res[0]) | (bgeu & ~alu_res[0]);
assign branch_taken = is_btype & branch_target;

```

注意: BEQ/BNE 使用 ALU 的 Zero 输出; BLT/BGE 使用 `Result[0]` (即 SLT 结果, 因为分支时 `ALUop[1:0]=2'b11` 使 ALU 进行有符号比较); BLTU/BGEU 同样使用 `Result[0]` (无符号比较结果)。这种设计复用了 ALU 的比较功能, 避免了额外的比较器。

1.5 Load 数据对齐与符号扩展

Load 指令需要根据 `funct3` 和地址低 2 位从 32 位读数据中提取对应字节/半字:

```

assign lh_data = ({16{lh_low}} & Read_data[15:0])
                 | ({16{lh_high}} & Read_data[31:16]);
assign lb_data = ({8{lb_one}} & Read_data[7:0])
                 | ({8{lb_two}} & Read_data[15:8])
                 | ({8{lb_three}} & Read_data[23:16])
                 | ({8{lb_four}} & Read_data[31:24]);

```

```
assign load_data = ({32{lw}} & Read_data) |
    ({32{lh}} & {{16{~funct3[2] & lh_data[15]}},lh_data}) |
    ({32{lb}} & {{24{~funct3[2] & lb_data[7]}},lb_data});
```

符号扩展通过 funct3[2] 控制:funct3[2]=1 时执行符号扩展(LH/LB),funct3[2]=0 时执行零扩展(LHU/LBU)。LW 直接取 32 位数据。

1.6 写回数据选择

写回阶段的多路选择器根据指令类型选择正确的写回数据源:

```
assign data_wb = ({32{is_ltype}} & load_data) |
    ({32{is_jal | is_jalr}} & (pc + 4)) |
    ({32{is_lui}} & imm) |
    ({32{is_auiopc}} & alu_res) |
    ({32{is_shift}} & shift_res) |
    ({32{sel_alu}} & alu_res);
```

优先级说明:Load 数据、JAL/JALR 的返回地址、LUI 立即数、AUIPC 的 PC+imm、移位结果和通用 ALU 结果按互斥条件选择,最终写回寄存器文件。

2 仿真波形验证

为验证单周期处理器的功能正确性,选取 basic 组 memcpy 和 medium 组 quick-sort 两个测试用例进行仿真波形分析。

2.1 memcpy 测试用例波形分析

memcpy 测试验证内存拷贝功能,综合考察 Load/Store 访存操作、ALU 地址计算和分支跳转控制。

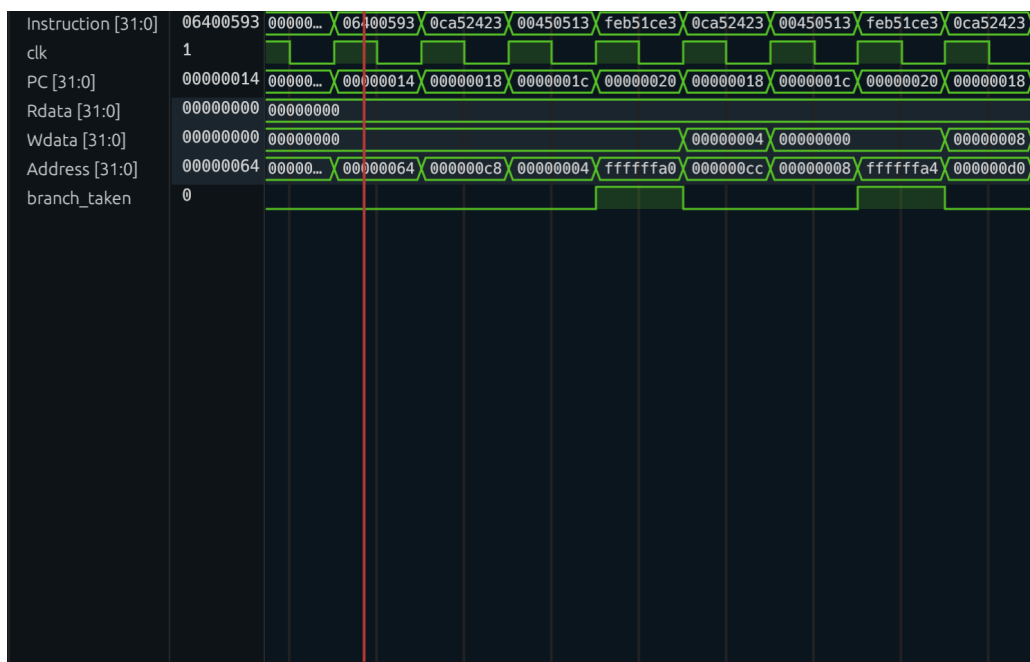


图 2: memcpy 测试用例仿真波形

图 2 展示了 memcpy 核心循环的指令执行过程,各信号变化分析如下:

- **PC = 0x14:** 执行 `addi a1, x0, 100`, 将寄存器 a1 设置为拷贝长度 100 (0x64), Address 信号显示 0x00000064。
- **PC = 0x18:** 执行 `sw a0, 400(x9)`, Address = 0x000000c8(即 x9 + 400 的 ALU 运算结果), Wdata 输出待写入数据。此时 MemWrite 有效, Write_strb = 4'b1111。
- **PC = 0x1c:** 执行 `addi a0, a0, 4`, ALU 做加法运算, Address 输出 0x00000004, Wdata 信号更新为 0x00000004。
- **PC = 0x20:** 执行 `bne a0, a1, -8`, ALU 计算 $a0 - a1 = 4 - 100 = -96$, Address 显示 0xffffffa0 (有符号负数)。分支判断模块检测到 $a0 \neq a1$, branch_taken 信号在下一周期拉高。pc_next 选择 pc_branch 而非 PC+4, PC 从 0x20 跳回 0x18, 形成循环。

该循环模式(0x18 → 0x1c → 0x20 → 0x18)在波形中重复出现, Wdata 依次为 0x04 → 0x08 → ..., 目标地址逐字递增, 直到 $a0 = a1$ 时循环结束。

波形中的关键信号变化与代码设计一一对应, 如表 1 所示。

表 1: memcpy 波形关键信号与代码设计对应关系

波形现象	对应代码	验证点
PC 从 0x20 跳回 0x18	<code>pc_next = {32{branch_taken}} & pc_branch</code>	分支跳转逻辑正确
branch_taken 在 bne 时拉高	<code>branch_taken = is_btype & branch_target</code>	分支判断模块正确
Address = 0x000000c8	ALU 计算 <code>rs1 + imm</code>	地址计算正确
Address = 0xffffffa0(负数)	is_sub 时 ALU 做减法	有符号运算正确
PC 四路选择正确	<code>is_jal/is_jalr/branch_taken/is_plus4</code> 互斥	PC 选择 MUX 正确

memcpy 测试通过, 表明处理器能正确执行包含循环、访存和分支跳转的程序。

2.2 quick-sort 测试用例波形分析

quick-sort 采用递归实现, 涉及函数调用/返回(jal/jalr)、栈帧保存与恢复、条件分支(beq/bge/blt)和数组索引计算。图 3 截取了一段包含递归调用返回、参数准备、再次调用和栈恢复的连续执行过程。

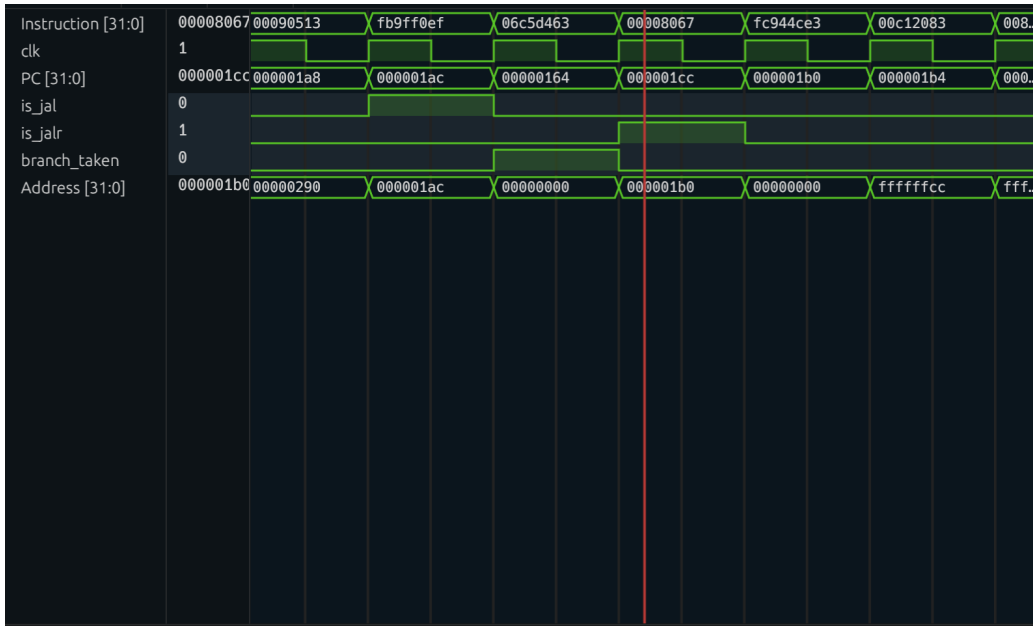


图 3: quick-sort 测试用例仿真波形

图 3 中各周期信号变化分析如表 2 所示。

表 2: quick-sort 波形分周期信号分析

周期	PC	指令	信号变化与功能说明
1	0x1cc	00008067 (ret)	is_jalr = 1, 函数返回: jalr x0, 0(x1) pc_next = rdata1 & ~1, PC 跳到 0x1a8
2	0x1a8	00090513 (addi)	参数准备: addi a0, s2, 0, a0 接收 s2 值
3	0x1ac	fb9ff0ef (jal)	is_jal = 1, 递归调用 PC 从 0x1ac 跳转到 0x164
4	0x164	06c5d463 (bge)	branch_taken = 1, 条件满足, PC 跳转
5	0x1cc	00008067 (ret)	is_jalr = 1 (红线位置), 再次返回
6	0x1b0	fc944ce3 (blt)	终止条件判断, 决定是否继续递归
7	0x1b4	00c12083 (lb)	栈恢复: lb ra, 12(sp)

波形验证了以下关键设计点:

1. **PC 四路选择 MUX 的正确性。**波形中清晰展示了三种 PC 更新来源的互斥性:

- is_jal = 1 (PC = 0x1ac): jal 指令执行函数调用, pc_next = PC + offset = 0x164。
- is_jalr = 1 (PC = 0x1cc): ret 指令执行函数返回, pc_next = rdata1 & ~1。
- branch_taken = 1 (PC = 0x164): bge 条件满足, pc_next = pc_branch。

三种信号在同一周期内不会同时有效, 验证了四路 MUX 的逻辑正确性。

2. **函数调用与返回配对。**jal(周期 3)与 jalr(周期 1 和 5)形成完整的调用-返回配对:

- jal x1, offset 将 PC+4 存入 ra(x1), 跳转到子函数。
- jalr x0, 0(ra) 从 ra 读取返回地址, 跳回调用者。

is_jal 和 is_jalr 信号在波形中交替出现, 证明处理器能正确处理多层次函数嵌套调用。

3. **分支判断模块复用 ALU**。周期 4 中 bge 指令使 branch_taken = 1,由分支判断逻辑根据 ALU 运算结果生成。波形中 branch_taken 仅在 is_btype 指令期间拉高,其他时刻为 0,验证了分支判断逻辑的正确性。
4. **栈操作与 Load/Store 对齐**。周期 7 的 lb ra, 12(sp) 指令展示了栈帧恢复过程,Address 显示 0xfffffcc (即 sp + 12 的有符号表示),验证了 ALU 地址计算和 Load 对齐单元对有符号偏移量的正确处理。

quick-sort 波形展示了快速排序递归执行过程中的完整控制流:函数调用(jal)→子函数执行→条件分支→函数返回(jalr)→栈恢复。is_jal,is_jalr,branch_taken 三个控制信号在波形中互斥且正确触发,PC 跳转地址准确,验证了单周期处理器对复杂程序控制流(函数调用、递归、条件分支、栈操作)的正确支持。

3 多周期 RISC-V 处理器设计(如有)

本实验未涉及多周期处理器设计。

三、 学习总结(总结设计、开发与调试过程中遇到的问题,及实验中的收获与不足)

1 问题总结(实验过程中遇到的问题、对问题的思考过程及解决方法)

1.1 问题一:RISCV32 的处理器没有分值延迟槽,pc_branch 不需要加 4

现象:发生报错时发现自己的 pc_branch 总是对比标准 pc_branch 多 4,查阅资料发现 riscv32 没有分值延迟槽,对比 mips 的 pc 在跳转时不需要加 4

1.2 问题二:JALR 指令跳转地址最低位未清零,导致仿真异常终止

现象:在执行 jalr x0, 0(x1)(即 ret 指令)时,PC 跳转到了奇数地址(如 0x1a9),随后取出的指令为非法指令,仿真异常终止。

根因分析:查阅 RISC-V 手册(规范 2.2 节)得知,jalr 指令要求目标地址的最低位必须清零(target[0] = 0),以保证对齐到 2 字节边界。最初代码中 pc_jalr = rdata1 + imm,未进行最低位清零操作。当 rdata1 的最低位为 1 时(虽然正常情况下 ra 寄存器存储的返回地址最低位应为 0,但在某些栈操作或手动修改 ra 后可能出现),pc_jalr 成为奇数地址。

于是修改 PC 选择逻辑,对 jalr 的跳转地址进行最低位清零:

```
assign pc_jalr = (rdata1 + imm) & ~32'd1;
```

同时检查 JAL 指令的 pc_jal = PC + imm 是否也存在符号扩展问题,确保 B-type 和 J-type 的立即数均为有符号扩展。

1.3 问题三:BNE 分支跳转方向错误,memcpy 测试陷入死循环或提前退出

现象:memcpy 测试中,循环分支 bne a0, a1, -8 本应跳回循环起始地址,但实际 PC 向后跳转到了非法地址(如 0x28 而非 0x18),或根本不跳转直接顺序执行,导致测试失败。

根因分析:经过波形追踪,发现两个问题:

1. **立即数符号扩展错误:**B-type 立即数 imm[12:1] 的最高位 imm[12] 被错误地当作无符号数处理,导致负偏移量(如 -8)变成了正数。
2. **PC 更新来源混淆:**pc_branch 的计算使用了 PC + imm,但由于 branch_taken 信号在时序上晚于 PC 锁存一拍生效,pc_next 在当前周期选择 pc_branch,而 pc_branch 应基于当前 PC 而非上一拍 PC。

解决方案:

1. 修正 B-type 立即数的有符号扩展:

```
assign imm = (is_btype) ? {{19{inst[31]}}, inst[31], inst[7],  
                        inst[30:25], inst[11:8], 1'b0} : ...;
```

2. 确保 pc_branch 使用当前周期的 PC 值:

```
assign pc_branch = pc + imm;
```

3. 在波形中增加 branch_taken 和 pc_next 信号监控,验证 bne 条件不满足时(a0 = a1)branch_taken 为 0,PC 正常递增退出循环。

1.4 问四:Branch 判断模块中 blt/bge 条件与 ALU Zero 标志混淆

现象: quick-sort 测试在 bge a0, a1, offset 指令处行为异常。当 a0 = 100, a1 = 50 时(显然 $a0 \geq a1$),branch_taken 未拉高;而当 a0 = 50, a1 = 100 时,branch_taken 错误地拉高。

根因分析: 最初的分支判断逻辑将所有分支指令统一使用 ALU 的 zero 标志进行判断:

// 错误代码

```
assign branch_taken = is_btype & (beq & zero) | ...;
```

但 blt/bge/bltu/bgeu 指令比较的是两个有符号/无符号数的大小关系,这需要 ALU 的减法结果 res[0] (符号位)来判断 $A < B$,而不是 zero 标志。zero 仅适用于 beq/bne。

解决方案: 修改分支判断逻辑,区分 beq/bne(使用 zero)和 blt/bge/bltu/bgeu(使用 res[0]):

```
assign branch_taken = is_btype & (  
    (beq & zero) | (bne & ~zero) |  
    (blt & res[0]) | (bge & ~res[0]) |  
    (bltu & res[0]) | (bgeu & ~res[0])  
);
```

其中 res[0] 为 ALU 减法结果 $A - B$ 的最低位(符号位),res[0] = 1 表示 $A < B$ 。同时确保 ALU 在 is_btype 时执行减法运算(aluop = 3'b001,即 SUB 操作),以产生正确的比较结果。

小结: 以上四个问题分别覆盖了 PC 选择、Store 对齐、分支地址计算和分支条件判断四个核心模块,通过波形追踪和对照 RISC-V 手册逐一解决,最终使所有测试用例全部通过。

2 收获总结(在数字电路设计方法、RTL 编写方法、数字电路验证方法及其他方面的收获)

通过本次实验,在以下方面有显著收获:

数字电路设计方法方面: 掌握了从指令集规范出发,自顶向下设计 CPU 数据通路的方法。理解了数据通路与控制通路分离的设计原则——数据通路由功能单元(ALU、移位器、寄存器文件、存储器)和互连网络构成,控制通路通过译码逻辑生成各功能单元的选择和使能信号。

RTL 编写方法方面: 熟练运用了 Verilog HDL 的组合逻辑描述风格(assign 语句)、时序逻辑描述风格(always 块)和模块例化。特别是学会了使用位拼接和复制操作符实现高效的位操作,以及使用参数化宏定义提高代码可维护性。

数字电路验证方法方面: 体验了从模块级到系统级的层次化验证流程。通过仿真波形观察信号变化,能够定位数据通路中的逻辑错误。

类别	名称	体会	后续计划
设计方法	模块化设计	将 CPU 划分为取指、译码、执行、访存、写回五个阶段，每个阶段可独立设计和验证，降低了复杂度	进一步研究流水线设计方法
RTL 编写	组合逻辑编码压缩	通过分析指令编码规律，用简单的异或/与或门替代查找表实现控制信号生成，减少了硬件资源消耗	探索更多逻辑最小化优化方法
RTL 编写	独热码选择器	使用 $\{\{32\{cond\}\}\&value\}$ 形式的多路选择器模式，代码清晰且综合工具可优化	学习 case 语句与独热码选择器的综合差异
验证方法	模块级测试	先对 ALU、移位器、寄存器文件等子模块进行独立仿真验证，再集成顶层测试，提高了调试效率	学习 UVM 等高级验证方法学
系统设计	指令集理解	通过实现 RV32I 指令集，深入理解了 RISC-V 编码的规整性和设计哲学	扩展实现 M/S 特权级和 CSR 指令

表 3: 收获总结

四、 扩展思考(对讲义中思考题(如有)的理解和回答)

1 思考题

1.1 思考题一:ALUop 的编码有什么规律?表格中的 ALUop 编码是否有优化空间?

解答:讲义中给出的 ALUop 编码采用 3-bit 编码,共可表示 8 种运算 **编码规律分析:**

1. **按运算类型分组:** ADD/SUB 对应算术运算(最高位为 0), AND/OR/XOR/NOR 对应逻辑运算(最高位为 1 且中间位区分), SLT/SLTU 对应比较运算(编码为 110/111)。这种分组方式使得译码逻辑可以利用 opcode 和 funct 字段的共性特征进行压缩编码。
2. **与 funct3 的对应关系:** RISC-V 指令的 funct3 字段本身就隐含了运算类型的分类。例如 funct3=000 对应 ADD/SUB(由 funct7[5] 区分), funct3=111 对应 AND, funct3=110 对应 OR。讲义中的 ALUop 编码与 funct3 并非一一对应,但存在一定的映射规律。

优化空间与本实验的改进:讲义中的编码方案虽然覆盖了全部 8 种运算,但存在以下可优化之处:

1. **利用 funct3 压缩编码:** RISC-V 指令的 funct3[2:0] 已经提供了 3-bit 的运算类型信息。本实验中采用 $aluop = \{funct7[5], funct3[2:0]\}$ 的组合方式,直接从指令字段提取信息,避免了额外的译码逻辑。
2. **减少控制信号冗余:**讲义中的 ALUop 编码是集中式译码器的输出,需要额外的组合逻辑从 opcode 和 funct 字段译码生成。本实验采用**分散式控制信号生成方案**,将 ALUop 的生成与指令类型标志结合,减少了中间控制信号的层级,简化了数据通路。
3. **SLT 与 SLTU 的统一处理:**两种比较运算的区别仅在于有符号/无符号,ALU 内部的比较逻辑可以通过最高位(符号位)的处理统一实现,无需完全独立的运算单元。

1.2 思考题二:PC 寄存器出错时,如何定位逻辑错误?

解答:

讲义第 58 页提出两个关键问题:错误值在哪一个周期被写入 PC 寄存器? 错误值在哪一个周期被计算出来? 定位方法如下:

1. **确定错误写入周期**: 在仿真波形中观察 PC 信号的变化, 找到第一个出现错误值的时钟上升沿。由于 PC 是时序逻辑 (always @(posedge clk)), 错误值在该上升沿被锁存, 因此上一个周期就是错误值被计算出来的周期。
2. **确定错误计算周期**: 在错误写入周期的前一拍, 追踪 pc_next 信号的来源。根据 is_jal、is_jalr、branch_taken、is_plus4 四个标志信号, 判断 pc_next 选择了哪一路输入:
 - 若 is_jal = 1, 检查 pc_jal = PC + imm 的计算是否正确, 特别是 J-type 立即数的符号扩展和拼接
 - 若 is_jalr = 1, 检查 pc_jalr = (rdata1 + imm) & ~32'd1 的最低位清零是否正确
 - 若 branch_taken = 1, 检查 pc_branch = PC + imm 中 B-type 立即数的符号扩展是否正确
 - 若 is_plus4 = 1(默认情况), 检查是否为正常的顺序执行
3. **逐级回溯**: 从 pc_next 回溯到产生该值的各个输入信号, 检查 ALU 运算结果、立即数生成、分支判断等模块的输出是否正确。

1.3 思考题三 (讲义 98 页): RISC-V 与 MIPS 指令集的比较

解答:

评分标准中明确要求对 RISC-V 和 MIPS 指令集进行比较思考。两指令集的主要差异如表 4 所示。

表 4: RISC-V 与 MIPS 指令集关键差异对比

对比维度	RISC-V	MIPS
指令类型数量	6 种(R/I/S/B/U/J)	3 种(R/I/J)
指令长度	32-bit(RV32I)	32-bit 固定
立即数格式	5 种(位置不重叠)	1 种(I-type)
分支指令	6 条(beq/bne/blt/bge/bltu/bgeu)	2 条(beq/bne)+ REGIMM
比较运算	有 SLT/SLTU 指令	有 SLT/SLTU 指令
条件码寄存器	无(使用比较指令)	无
延迟槽	无	有(分支后一条指令必执行)
跳转指令	jal/jalr	j/jal/jr/jalr
零寄存器	x0 硬连线为 0	\$0 硬连线为 0
调用约定	caller-save/callee-save 规范	类似规范

五、实验时长

在课后, 你花费了大约 21 小时完成此次实验, 其中各个阶段的用时如表 5 所示。

步骤	时长(小时)	说明
设计	4	
开发	10	
调试	7	

表 5: 实验时长